

OOP MINI PROJECT

Patient Medical History Visualization

Project Title: Patient Medical History Visualization

Course: Object-Oriented Programming using Java

Submitted by: Lakshaa v.s

Register Number: 2117240070164

Department: B.tech Artificial intelligence and data science

Institution: Rajalakshmi institute of technology

Academic Year: 2024 – 2025

Submitted to: Mr. Satheesh

2. Introduction

Healthcare systems require an efficient way to record and analyze patient data for diagnosis and treatment. Manual tracking of patient medical history can lead to errors and inefficiency.

The Patient Medical History Visualization project aims to build a Java-based application that stores, retrieves, and visualizes patients' health records using Object-Oriented Programming (OOP) concepts such as classes, objects, inheritance, encapsulation, and polymorphism.

This system allows doctors and patients to visualize trends in health parameters such as blood pressure, sugar levels, and temperature using simple graphs or textual summaries.

3. Objectives

- To implement a Java-based application using OOP principles.
- To maintain a structured medical history for each patient.
- To display health data visually using text-based charts or JavaFX.
- To apply real-world use of classes, inheritance, and data encapsulation.

To provide a simple and efficient health data management tool.

4. System Requirements and Setup

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: 4 GB or more
- Storage: Minimum 500 MB free space

Software Requirements

- Operating System: Windows / macOS / Linux
- IDE: IntelliJ IDEA / Eclipse / NetBeans
- JDK Version: JDK 11 or higher
- Libraries: JavaFX (for visualization), Collections Framework

Setup Instructions

1. Install Java Development Kit (JDK).
2. Install IDE such as IntelliJ or Eclipse.
3. Download and set up JavaFX SDK (if used).
4. Clone or import the project files.
5. Run the Main.java file to start the program.

5. Procedure

1. Design Classes:

Create classes for Patient, MedicalRecord, and Visualizer.

2. Encapsulate Data:

Store patient information (ID, name, age, medical data) using private attributes.

3. Use Methods:

Define methods for adding, displaying, and updating patient records.

4. Apply Inheritance:

Inherit visualization functionality through a subclass like Visualizer extending Patient.

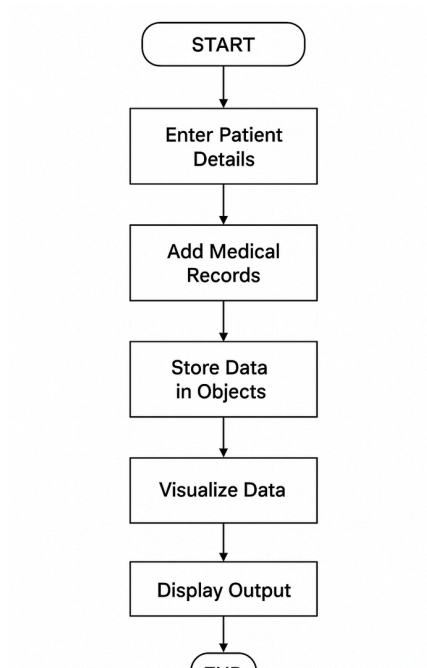
5. Implement Visualization:

Generate simple text-based or GUI-based charts (JavaFX or console output).

6. Output Results:

Display stored records and graphical health trends.

6. Flow Diagram



7. Code

```
import java.util.ArrayList;

class Patient {

    private String patientID;

    private String name;

    private int age;

    public Patient(String patientID, String name, int age) {

        this.patientID = patientID;

        this.name = name;
```

```

        this.age = age;
    }

    public void displayPatientInfo() {

        System.out.println("Patient ID: " + patientID);

        System.out.println("Name: " + name);

        System.out.println("Age: " + age);

    }

}

class MedicalRecord {

    private ArrayList<String> dates = new ArrayList<>();

    private ArrayList<Integer> bpLevels = new ArrayList<>();

    private ArrayList<Integer> sugarLevels = new ArrayList<>();

    public void addRecord(String date, int bp, int sugar) {

        dates.add(date);

        bpLevels.add(bp);

        sugarLevels.add(sugar);

    }

    public void displayRecords() {

        System.out.println("\nMedical History:");

        for (int i = 0; i < dates.size(); i++) {

            System.out.println("Date: " + dates.get(i) +

                " | BP: " + bpLevels.get(i) +

                " | Sugar: " + sugarLevels.get(i));

        }

    }

}

```

```

    }

    public ArrayList<Integer> getBpLevels() { return bpLevels; }

    public ArrayList<Integer> getSugarLevels() { return sugarLevels; }

    public ArrayList<String> getDates() { return dates; }

}

```

```

class Visualizer {

    public void visualize(MedicalRecord record) {

        System.out.println("\n--- Visualization of Health Trends ---");

        ArrayList<String> dates = record.getDates();

        ArrayList<Integer> bp = record.getBpLevels();

        ArrayList<Integer> sugar = record.getSugarLevels();

        for (int i = 0; i < dates.size(); i++) {

            System.out.print(dates.get(i) + " | BP: ");

            for (int j = 0; j < bp.get(i) / 10; j++) System.out.print("*");

            System.out.print(" (" + bp.get(i) + ")");

            System.out.print(" | Sugar: ");

            for (int j = 0; j < sugar.get(i) / 10; j++) System.out.print("#");

            System.out.println(" (" + sugar.get(i) + ")");

        }

        // Calculate and display average values

        double avgBP = 0, avgSugar = 0;

        for (int i = 0; i < bp.size(); i++) {

            avgBP += bp.get(i);

```

```

        avgSugar += sugar.get(i);
    }

    avgBP /= bp.size();

    avgSugar /= sugar.size();


    System.out.println("\nAverage BP: " + String.format("%.2f", avgBP));

    System.out.println("Average Sugar Level: " + String.format("%.2f", avgSugar));

}

}

public class Main {

    public static void main(String[] args) {

        Patient p1 = new Patient("P001", "Lakshaa", 21);

        MedicalRecord record = new MedicalRecord();


        record.addRecord("2025-01-10", 120, 90);

        record.addRecord("2025-02-12", 118, 95);

        record.addRecord("2025-03-20", 115, 85);


        p1.displayPatientInfo();

        record.displayRecords();


        Visualizer viz = new Visualizer();

        viz.visualize(record);

    }

}

```

8. Screenshots

```
Output
Patient ID: P001
Name: Lakshaa
Age: 21

Medical History:
Date: 2025-01-10 | BP: 120 | Sugar: 90
Date: 2025-02-12 | BP: 118 | Sugar: 95
Date: 2025-03-20 | BP: 115 | Sugar: 85

--- Visualization of Health Trends ---
2025-01-10 | BP: ***** (120) | Sugar: ##### (90)
2025-02-12 | BP: ***** (118) | Sugar: ##### (95)
2025-03-20 | BP: ***** (115) | Sugar: ##### (85)

Average BP: 117.67
Average Sugar Level: 90.00

=== Code Execution Successful ===
```

9. GitHub Reference

Repository Link:

<https://github.com/lakshaa26/PatientMedicalHistoryVisualizationJava>